

Coaster Detection Engine and Multi-User Touch UI for Bar Environments

HUNG Ka Hin
20962096
khhungac@connect.ust.hk

Abstract— Background: *Tales9* is a 1900×1900 smart bar table piloting at Barcode, Hong Kong. The monitor has no native touch; input arrives via an infrared overlay (40+ points, USB). Each coaster has three mechanical pillars registering as distinct touch points; identity follows pillar geometry. Guests use finger touch on the same surface. Bar constraints: spills, condensation, dim light, loud music, impaired motor control. **Intended outcomes:** (1) Identify coasters by geometric signature with sub-100 ms feedback; (2) four concurrent users without panel-pillar interference; (3) preview→confirm detection; (4) UI tolerant of imprecise multi-point touch. **Application design:** *InputAdapter* ingests IR frames; *TrackingEngine* clusters points, scores triangle signatures, emits lifecycle events. *AnimationDispatcher* reserves full animations for confirmed identity. *User Nodes* use edge-aware orientation, 16 px tap slop, pointer-up handlers. Constants tuned from 10–20 live captures per coaster. **Findings:** A five-stage pipeline—clustering, scale-invariant features, template matching, two-frame confirmation, long removal debounce—reduced false triggers while preserving sub-100 ms preview. Empirical tolerance tuning (cluster radius 0.08→0.13) mattered more than algorithmic sophistication. Routing panel input away from the common-space surface restored menu usability.

Keywords—coaster detection, geometric signature, infrared touch overlay, three-pillar coaster, template matching, scale-invariant descriptor, DBSCAN clustering, temporal hysteresis, multi-touch tracking, preview-confirm lifecycle

I. ABOUT THE PROJECT (PROBLEM-SOLUTION) AND THE PART YOU ARE RESPONSIBLE FOR

TALES9 turns a bar table into an interactive social surface. Beneath tempered glass: a passive monitor and an IR touch overlay serving as the sole input plane. Up to four guests receive colour-coded User Nodes for menus, quizzes, and orders; bartenders pair drinks to coasters via NFC or printed QR; when a coaster lands, its three pillars greet the overlay and animations appear in the Common Space. As Head of Software, I built the detection and interaction core—*TrackingEngine*, *CoasterTemplates*, *CalibrationMapper*, *InputAdapter*—plus the multi-user panel system (*UserNode*, edge orientation, tap-vs-drag classification, *usePressAction*) and the render pipeline honouring preview vs confirmed states (*AnimationDispatcher*, *PixiStage*). I integrated Firebase order/coaster sync and the *DebugPanel* used during on-site calibration. Teammates will handle IP54 enclosure and glass stack; I ensured the software respected what physics actually does. Salet6 (AI venue branding) extends *Tales9* theming but sits outside this report. Here, the subject is detection, multi-user touch, and designing for environments where the user manual ought to include “may contain alcohol.”

II. KNOWLEDGE AND RESEARCH FOUNDATION (DEPTH OF KNOWLEDGE AND CLARITY IN DEFINING KEY CONCEPTS. RELEVANCE AND QUALITY OF BACKGROUND RESEARCH CONDUCTED.)

A. Geometric Signatures on Infrared Touch Overlays

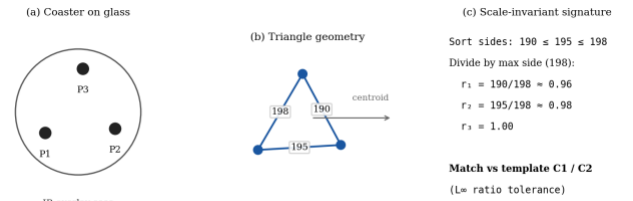


Fig. 1. Three mechanical pillars on tempered glass register as IR touch points; side lengths form a triangle whose sorted, normalised ratios identify coaster templates C1 and C2.

Commercial IR overlays detect touch by beam interruption along an LED/sensor frame; opaque objects above a minimum diameter (often ~5–8 mm) register as coordinate hits with typical accuracy around ± 2 mm [1][2]. They report multi-touch points, not object identities—unlike marker-based tangible systems (e.g. fiducial or AI-trained object modules [3]). *Tales9* therefore implements application-layer identification: each coaster’s three mechanical pillars press the glass and appear as three IR contacts; identity is inferred from triangle geometry, not from the overlay itself (NFC and printed QR remain bartender-side only). Scale-invariant shape descriptors are a standard pattern-recognition tool: normalising by size or using moment invariants lets templates match objects at unknown distance or pressure [4]. We adopt a lightweight variant: sort the three side lengths, divide by the longest side, and obtain a ratio triple invariant to uniform scale and rotation (vertex order). Position is the triangle centroid mapped to 1900×1900 px via *CalibrationMapper*. This is less general than Hu moments [4] but sufficient for a closed set of two coaster prototypes and cheap enough for per-frame execution on bar hardware.

B. Staged Response and Bar-Environment UX

Guests want magic; order state wants evidence. Our preview→confirm→cancel model maps to *preview_started* → confirmed → removed. Temporal hysteresis—requiring stable observations before committing state—is widely used in tracking to reject single-frame noise [5]; our two-frame confirm gate is a minimal form of this. Barcode operation assumes low light, high noise, spills, condensation, and impaired motor control. Fingers and pillars share one IR stream; mitigations include 16 px tap slop, pointer-up panel actions, edge-oriented panels, and a social ownership model (the overlay cannot identify fingers). IP54 hardware keeps electronics dry; software keeps functioning when the glass is not.

III. METHODOLOGY AND ANALYSIS (EXPLORATION OF DIFFERENT POSSIBLE SOLUTIONS AND APPROACHES. INSIGHTFUL ANALYSIS OF RESULTS AND THOUGHTFUL REFLECTIONS)

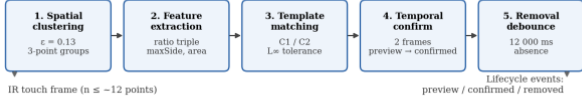


Fig. 2. Per-frame TrackingEngine pipeline from raw IR points to preview, confirmed, and debounced removal events.

A. Coaster Tracking Algorithm Design

The TrackingEngine implements a five-stage per-frame pipeline. Each stage exposes tunable parameters; values below are from TrackingEngine.ts and CoasterTemplates.ts, calibrated from 10–20 debug captures per coaster on live hardware. **Stage 1 — Spatial clustering.** Raw IR points (normalised 0–1) are grouped by pairwise distance: any set of three points whose members all lie within CLUSTER_RADIUS forms a candidate cluster. Current value: 0.13 (raised from 0.08 → 0.12 → 0.13 when real pillar spread on 1900 px canvas exceeded initial estimates). Algorithmically this resembles fixed-radius neighbour grouping, a simplified case of density-based clustering where $\epsilon = \text{CLUSTER_RADIUS}$ and $\text{minPoints} = 3$ [6]. Complexity is $O(n^3)$ on active points; acceptable because $n \leq \sim 12$ (four coasters $\times$ three pillars + fingers). **Stage 2 — Feature extraction.** For each 3-point cluster, signatureMetrics() computes: (a) sorted ratio triple with each side divided by max side; (b) maxSide — longest edge in normalised coordinates; (c) area — triangle area via the shoelace formula (rejects collapsed or near-collinear triangles). **Stage 3 — Template matching.** Each cluster is scored against prototypes C1 and C2, built from median fitted pillar spacings (coaster-1: 190/195/198, ratioTolerance 0.06; coaster-2: 138/175/179, ratioTolerance 0.07). A cluster qualifies only if ratio delta (L_∞ norm) $\leq$ tolerance, maxSide within scale range, and area within range. Non-qualifying clusters emit rejected. A unique-template constraint enforces at most one active match per template ID per frame. **Stage 4 — Temporal confirmation.** On first match, a coaster enters preview; after COASTER_CONFIRM_FRAMES = 2 stable frames, state upgrades to confirmed and business logic may fire. At typical IR report rates (≥ 60 Hz [1]), two frames sit inside our 16–80 ms confirmation target. **Stage 5 — Removal debounce.** Deletion occurs only after REMOVAL_DEBOUNCE_MS = 12 000 ms without observation, tolerating brief lift-and-sip gestures and IR dropouts from wet glass.

B. Multi-User Touch Routing

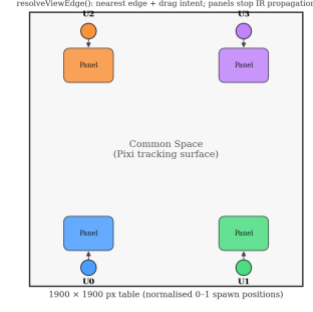


Fig. 3. User Nodes spawn at predefined edge positions; resolveViewEdge() orients panels away from the nearest table edge while tracking stays on the common-space surface.

Four users and four coasters can produce 12+ simultaneous IR contacts. Panel vs coaster conflict: root-level input misread finger taps as phantom coasters; we confined tracking to the Pixi common-space surface and stopped propagation on panels. Click synthesis failure: browsers declined to synthesize clicks while pillars held contacts; usePressAction and 16 px slop restored sanity. Sessions spawn 1–4 User Nodes at edge positions; resolveViewEdge() rotates panels toward readable edges; overlapping panels are tolerated in the MVP.

C. Results and Reflections

124 Vitest tests cover clustering, template rejection, two-template concurrency, temporal confirm, and debounced removal. On hardware, drink-arrival fires only after two confirmed frames. The highest-leverage tuning was CLUSTER_RADIUS and per-template tolerances fit to capture medians—confirming that embedded geometry problems reward field measurement over desk theory.

IV. EVALUATION OF THE RESULTS AND FUTURE STUDIES

The engine met MVP targets: identity and position for two coaster templates via three-pillar IR signatures, centroid-driven animation, 12 s removal debounce. Multi-user UI survived concurrent coasters. **Limitations:** naive first-three-point clustering if >3 contacts fall in one radius; no online learning; geometry alone cannot distinguish a pillar from an unusually cooperative finger; 12 s debounce delays cleanup; parameters are venue-specific. **Future work:** Firebase template overrides; k-medoids or combinatorial 3-point selection within dense clusters; Mahalanobis distance on feature vectors if the template library grows [4]; recorded IR trace replay in CI; spill-condition regression suites.

V. REFERENCES

- [1] GROOVY Technology Corp., "Infrared Touch Technology Overview," Gtouch product documentation, accessed May 2026.
- [2] Sony Professional, "Multi-Touch Overlay Kit (STP-IR100 Series) Specifications," product datasheet, accessed May 2026.
- [3] Interactive Scape GmbH, "Scape X Engine — Object Recognition for Multi-Touch Displays," product documentation, accessed May 2026.
- [4] M. K. Hu, "Visual Pattern Recognition by Moment Invariants," *IRE Trans. Information Theory*, vol. 8, no. 2, pp. 179–187, 1962.
- [5] J. L. Crowley and F. Berard, "Multimodal Tracking and Natural Interaction for the Perceptual Workbench," *Proc. IEEE Int. Workshop on Robot and Human Communication (ROMAN)*, 1997.
- [6] M. Ester et al., "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise," *Proc. KDD*, 1996, pp. 226–231.

VI. SELF-REFLECTION

A. *Was the final year project an adequate experience that summed up four years of ISD studies? How?*

No, not as a year 4 capstone, maybe Y3... If “summed up four years” means exercising the full ISD stack—embedded systems, actuators, sensors, motors, tight hardware–software co-design, and the kind of mechatronic depth other FYP teams shipped—Tales9 fell short of my expectations. Peer projects wrestled with microcontrollers, motion control, and richer physical I/O; our table’s sensing path is largely an IR touch overlay over USB, with coasters identified in software from pillar geometry.

I invested considerable effort learning embedded systems at HKUST; this FYP did not give me enough room to apply that. Hardware integration on my side was also limited: beyond the overlay stream, Firebase sync, and on-site calibration, there was little of the connectivity, motor, actuator, or sensor work that made earlier courses feel so valuable—and that I had hoped to carry into the final year.

That said, I am genuinely satisfied on the software engineering axis, and that part does feel like mature FYP work. I practiced OOP separation (TrackingEngine, AnimationDispatcher, InputAdapter), kept the codebase documented and testable (124 Vitest tests, CHANGELOG discipline), and worked in a test-driven workflow where engine logic could be verified without the physical table. Moving from JavaScript to TypeScript sharpened how I think about contracts and maintainability.

Tales9 solidified my experience as someone who can architect a system—not only implement screens—and I grew comfortable making trade-offs under constraint: preview before confirm, long removal debounce, panel vs common-space input routing, tolerances tuned from field captures rather than ideal models.

So the honest split is: strong Year 4 software capstone; incomplete Year 4 systems/embedded capstone. ISD taught me both; Tales9 rewarded the former more than the latter. For a project scoped to a smart-table kiosk and bar pilot, that was enough to ship—but it does not, by itself, close the loop on everything I paid tuition to learn.

B. *Final Thoughts*

Context is part of the specification—that lesson landed fully. An engine calibrated on dry glass fails when condensation shifts IR points; a UI built on click fails when four fingers and four three-pillar coasters share one overlay. I would record IR touch traces earlier and calibrate pillars on-site before declaring victory. If I choose embedded or mechatronic work next, I will need a project that actually uses those skills—not only borrows them in theory. If I choose software architecture, Tales9 was good preparation: constrained delivery, clear module boundaries, and the humility to tune constants until Friday night stops arguing back. Either path, the bar worth clearing is still the same—build something that survives contact with real users, not just the grading rubric.